

Implementação de um sistema de Content-Based Image Retrieval (CBIR) - Pós Processamento

Alunos: João A. Biava, Matheus Gonzaga, Luiz André da Mota e Pedro H. Stachuka
Passos

Link de acesso no Github: https://github.com/matt-gzg/recuperacao_imagens

Foi implementado um sistema de recuperação de imagens por conteúdo (CBIR) baseado em descritores locais e representação por vocabulário visual. Ele é composto por quatro etapas principais: carregamento dos dados, indexação dos documentos, execução das queries e visualização dos resultados.

Carregamento dos dados:

O dataset utilizado é o Caltech-101, carregado via ImageFolder do PyTorch. As imagens são divididas em dois conjuntos disjuntos: um de documentos, que forma a base indexada, e um de queries, cujas imagens nunca aparecem na base. Para cada categoria selecionada, as últimas n_{query} imagens são reservadas como queries e as demais compõem os documentos. Todas as imagens são redimensionadas para 224×224 pixels e convertidas para arrays NumPy RGB.

Indexação:

A indexação opera sobre regiões obtidas por uma grade 3×3 aplicada a cada imagem, gerando nove regiões por documento. Para cada região, descritores SIFT são extraídos. Após coletar todos os descritores de todos os documentos, um vocabulário visual é construído com K-means. Cada região é então representada por um histograma Bag of Visual Words (BoVW): cada descritor SIFT é atribuído ao centro mais próximo, e o histograma resultante é normalizado.

Execução das queries:

Para cada imagem de query, o usuário seleciona interativamente uma região de interesse. Descritores SIFT são extraídos e convertidos em um histograma BoVW usando o mesmo vocabulário construído na indexação. A busca compara este histograma com todos os histogramas do índice por dois critérios independentes: similaridade de cosseno entre histogramas BoVW, que captura semelhança de textura e estrutura local, e IoU entre a bbox da query e as bboxes das regiões indexadas, que prioriza candidatos na mesma posição espacial. Em ambos os casos, quando um documento possui múltiplas regiões candidatas, apenas a de maior score é mantida, e o ranking final retém os cinco melhores documentos.

Visualização:

Para cada query são geradas imagens mostrando o ranking por similaridade e o ranking por IoU, com bordas coloridas indicando acerto (verde) ou erro (vermelho) em relação à categoria da query.

Pós-Processamento:

- Filtro por formato;
- Re-ranking por agrupamento de clusters.

```
# Parâmetros de pós-processamento
LIMITE_DIFERENÇA_FORMATO = 0.15
N_CLUSTERS_POS = 5
BONUS_CLUSTER = 0.25
```

“LIMITE_DIFERENÇA_FORMATO” define a diferença máxima de circularidade aceita entre query e documento. “N_CLUSTERS_POS” é o número de grupos usados no re-ranking. “BONUS_CLUSTER” é o valor adicionado ao score dos candidatos que pertencem ao mesmo cluster da query.

Pós-processamento 1: Filtro por Formato

Para garantir que o ranking cubre todos os documentos antes do filtro, “buscar_query” é chamada novamente com “top_k=len(indice)”:

```
ranking_sim_amplo, _, hist_query = buscar_query(
    query_img, indice, centros,
    bbox_query=bbox_query,
    top_k=len(indice) # todos os documentos
)
mantidos_formato, _, circ_query = filtrar_por_formato(
    ranking_sim_amplo, query_img, bbox_query,
    limite_diferenca=LIMITE_DIFERENCA_FORMATO
)
ranking_formato = mantidos_formato[:5]
```

A função “calcular_circularidade” mede o formato do objeto pela circularidade do seu maior contorno ($4\pi \cdot \text{área} / \text{perímetro}^2$), que vale 1,0 para um círculo perfeito. O limiar de binarização é definido automaticamente pelo método de Otsu, tornando o cálculo mais robusto a variações de iluminação:

```
def calcular_circularidade(img_rgb, bbox=None):
    if bbox is not None:
        x, y, w, h = bbox
        recorte = img_rgb[y:y+h, x:x+w]
    else:
        recorte = img_rgb

    cinza = cv2.cvtColor(recorte, cv2.COLOR_RGB2GRAY)

    _, binaria = cv2.threshold(cinza, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)
    contornos, _ = cv2.findContours(binaria, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
```

```

if len(contornos) == 0:
    return 0.0

maior = max(contornos, key=cv2.contourArea)
area = cv2.contourArea(maior)
perimetro = cv2.arcLength(maior, True)

if perimetro == 0:
    return 0.0

return (4 * np.pi * area) / (perimetro ** 2)

```

A função “filtrar_por_formato” compara a circularidade da ROI selecionada na query com a detectada automaticamente em cada documento. Para a query usa-se a “bbox” exata do usuário. Para os documentos, analisa-se a imagem inteira, detectando o contorno do objeto principal. Candidatos cuja diferença de circularidade supera “LIMITE_DIFERENCA_FORMATO” são removidos:

```

def filtrar_por_formato(ranking, query_img, bbox_query,
limite_diferenca=LIMITE_DIFERENCA_FORMATO):

    circ_query = calcular_circularidade(query_img, bbox=bbox_query)

    mantidos = []
    removidos = []

    for item in ranking:
        # Imagem inteira do documento – sem bbox fixa de grid
        circ_item = calcular_circularidade(item["imagem"], bbox=None)
        diferenca = abs(circ_query - circ_item)
        item_annotado = {**item, "circularidade": circ_item,
"dif_formato": diferenca}

        if diferenca <= limite_diferenca:
            mantidos.append(item_annotado)
        else:
            removidos.append(item_annotado)

    print(f" [Filtro Formato] circularidade query={circ_query:.3f} | "
          f"mantidos={len(mantidos)} | removidos={len(removidos)}")

    return mantidos, removidos, circ_query

```

Pós-processamento 2: Re-ranking por Cluster

Opera sobre o ranking amplo para que o KMeans tenha candidatos suficientes para formar grupos significativos. Após o agrupamento e aplicação do bônus, corta no top 5:

```
ranking_cluster_amplo, cluster_query = reranking_por_cluster(
    ranking_sim_amplo, hist_query,
    n_clusters=N_CLUSTERS_POS,
    bonus=BONUS_CLUSTER
)
ranking_cluster = ranking_cluster_amplo[:5]
```

A função “reranking_por_cluster” agrupa os candidatos via KMeans sobre seus histogramas BoVW. O cluster ao qual o histograma da query pertence recebe um bônus no score final, promovendo candidatos visualmente similares à query. Quando o número de candidatos for menor que “N_CLUSTER_POS”, o valor de k é reduzido automaticamente para evitar erros:

```
def reranking_por_cluster(ranking, hist_query,
n_clusters=N_CLUSTERS_POS, bonus=BONUS_CLUSTER):

    if len(ranking) < n_clusters:
        # Menos candidatos que clusters: reduz k para evitar erro do
KMeans
        n_clusters = max(2, len(ranking))

    histogramas = np.array([item["histograma"] for item in ranking])

    kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
    clusters = kmeans.fit_predict(histogramas)
    cluster_query = kmeans.predict(hist_query.reshape(1, -1))[0]

    ranking_novo = []
    for item, cluster_id in zip(ranking, clusters):
        bonus_item = bonus if cluster_id == cluster_query else 0.0
        score_final = item["similaridade"] + bonus_item
        ranking_novo.append({
            **item,
            "cluster": int(cluster_id),
            "cluster_query": int(cluster_query),
            "bonus_cluster": bonus_item,
            "score_final": score_final
        })
```



```

ranking_novo.sort(key=lambda x: x["score_final"], reverse=True)

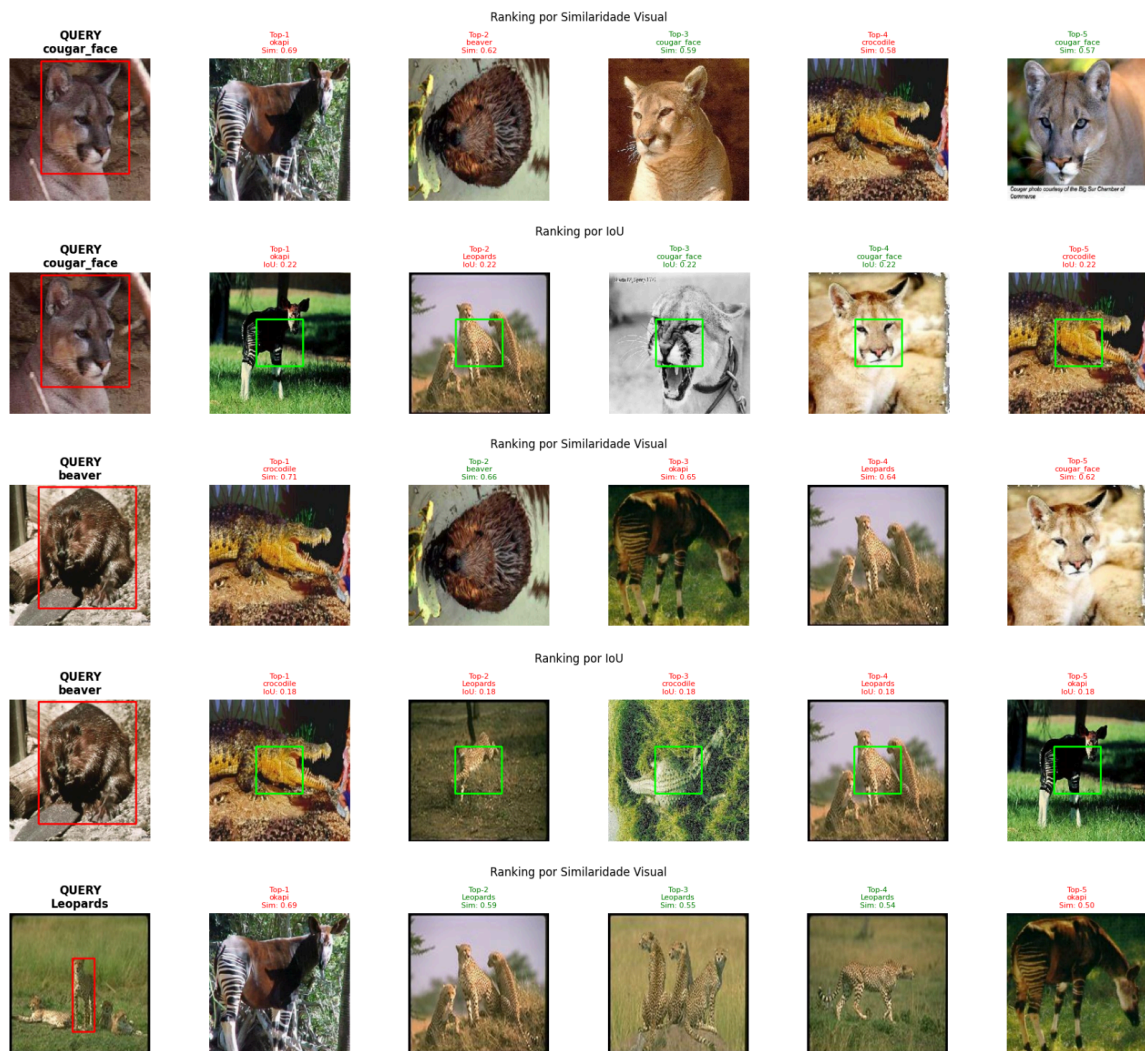
n_promovidos = sum(1 for i in ranking_novo if i["bonus_cluster"] >
0)

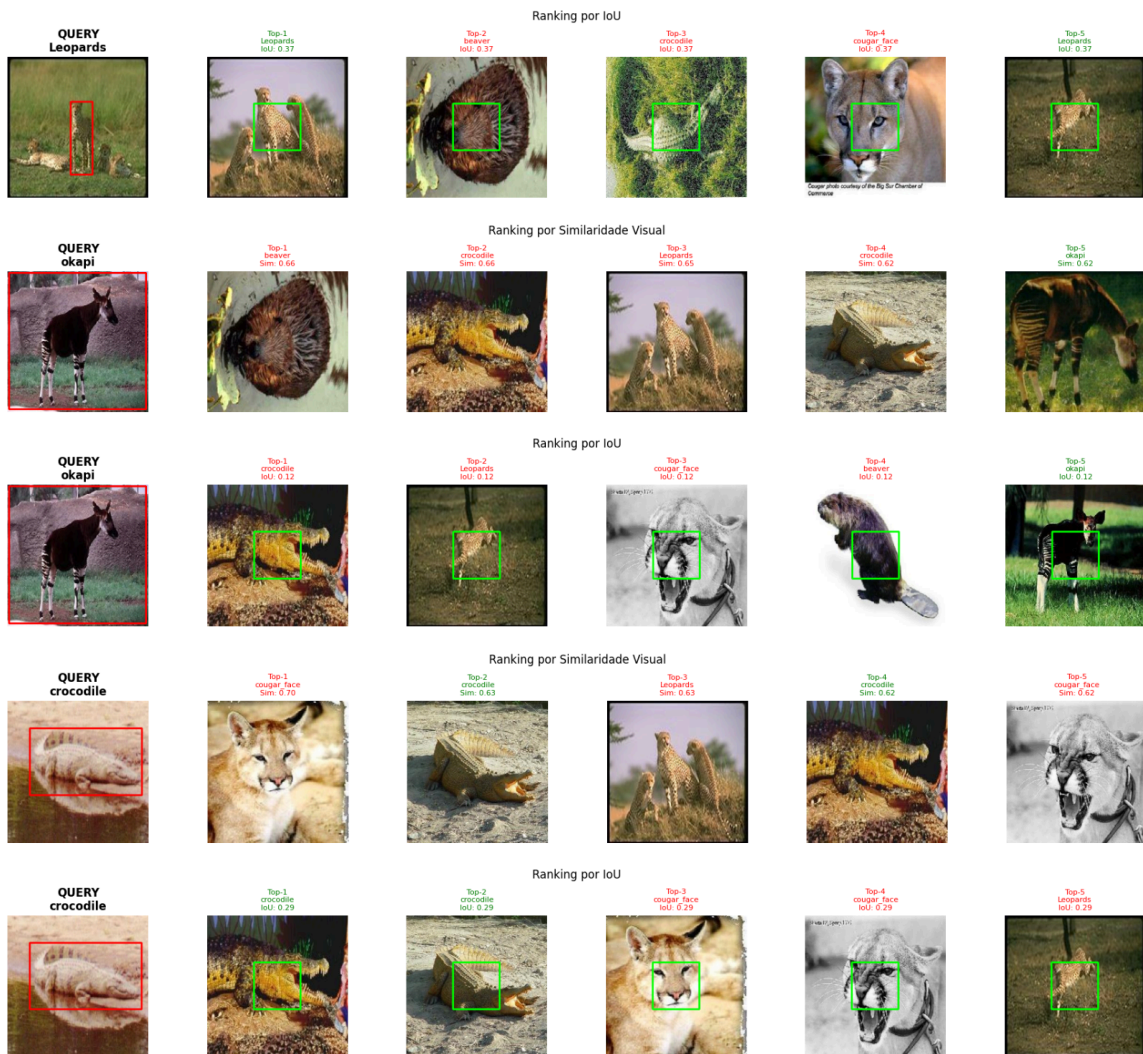
print(f" [Re-ranking Cluster] cluster query={cluster_query} | "
      f"  candidatos   promovidos={n_promovidos} /
{len(ranking_novo)}")

return ranking_novo, cluster_query

```

Resultados Anteriores (Similaridade de Cosseno e IoU)





Resultados após Filtro por Formato:

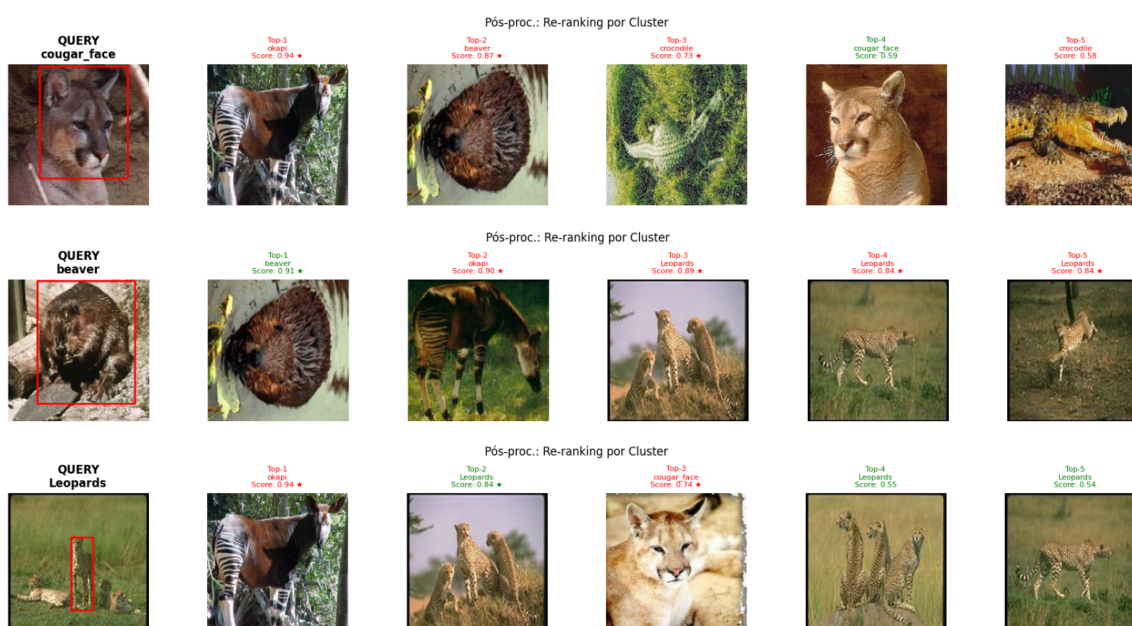


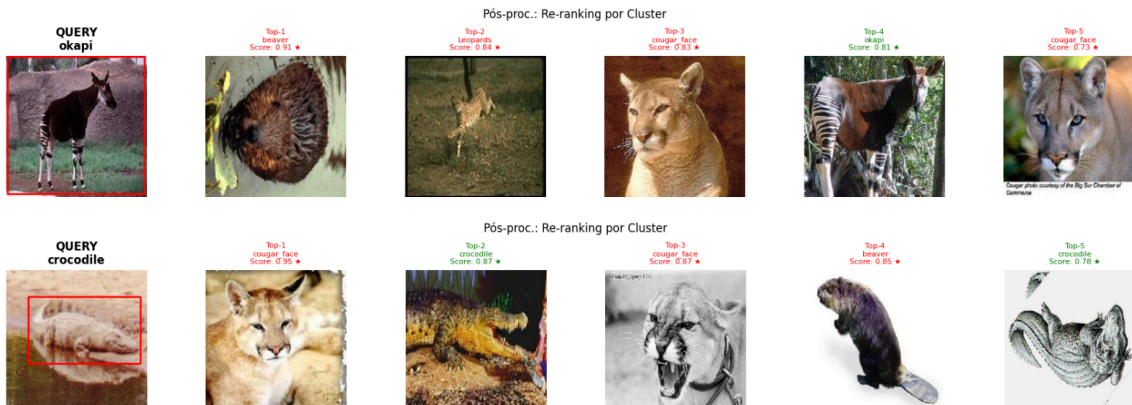


Os resultados mostram que o filtro por formato teve eficácia limitada no dataset utilizado, por conta das variações de pose, enquadramento e iluminação das fotografias do dataset, fazendo com que animais de categorias distintas produzam contornos com circularidades semelhantes. Por isso, os valores permanecem baixos mesmo entre categorias diferentes, e o filtro remove poucos candidatos.

Os resultados indicam que a circularidade isolada é insuficiente como critério de filtragem para imagens fotográficas com variação real, sendo mais adequada a cenários controlados com objetos bem delimitados, sem grandes variações de fundo, etc.

Resultados do Re-ranking por Cluster





Os resultados mostram desempenho variado entre as queries. Nos casos de beaver e Leopards o agrupamento identificou clusters com boa coesão visual, resultando em acertos consistentes no top-5. Na query de Leopards, três dos cinco resultados são da categoria correta, com scores elevados.

Nos demais casos o re-ranking não produziu melhora significativa. Categorias como cougar_face, okapi e crocodile compartilham características visuais globais, como coloração terrosa, textura de pelagem, que fazem com que suas imagens se agrupem no mesmo cluster, misturando categorias distintas e limitando a precisão do método.

O símbolo ★ indica candidatos promovidos pelo bônus de cluster. Em vários rankings candidatos incorretos recebem o bônus junto com os corretos, evidenciando que o cluster da query não é homogêneo em termos de categoria.